Development Effort and Investment Comparison: `xboing-python` vs `xboing-c`

XBoing Modernization Project

June 2026 — Generated from git history

Contents

1 Overview

This report compares the development effort invested in two ports of the classic XBoing (1993, Justin C. Kibell) breakout game:

- `xboing-python` (https://github.com/jmf-pobox/xboing-python) — Python 3.10+ / pygame.
- `xboing-c` (https://github.com/jmf-pobox/xboing-c) — C11 / SDL2.

Both projects share the same maintainer and the same goal: a faithful port of the original XBoing. They differ in language, timing, tooling, and the degree of AI-assisted development. This report quantifies those differences using git history as the primary data source and correlates development phases with the Claude model generations available at each point.

2 Timeline

	xboing-python	xboing-c
First commit	2025-05-03	2025-11-26
Latest commit	2026-06-02	2026-05-30
Calendar span	13 months	6 months
Active development days	24	30
Elapsed active weeks	6	16

Activity pattern. `xboing-python` had a single intense burst in May 2025 (301 commits in 4 weeks), then went dormant for 6 months, with brief activity in November 2025 (5 commits) and June 2026 (3 commits). `xboing-c` started 7 months later but sustained development across 16 weeks from February–May 2026, with a peak of 162 commits in February alone.

3 Volume and Velocity

3.1 Aggregate Metrics

Metric	xboing-python	xboing-c
Total commits	310	264
Merged pull requests	18	137
Lines added	64,568	142,861
Lines removed	21,066	22,933
Net lines	+43,502	+119,928
Source lines (current)	12,372	23,332
Test lines (current)	5,674	25,442
Test functions	202	1,115
Source files	82 (.py)	43 (.c)
Test files	40	59

Observation. `xboing-python` has more raw commits (310 vs 264) but far fewer PRs (18 vs 137). The Python repo's early history consists of direct pushes to master; the C repo adopted a strict PR workflow from the start, with every change going through review. The C repo's net line output is 2.75× higher, and its test corpus is 4.5× larger.

3.2 Monthly Breakdown

Month	Repo	Commits	Lines +	Lines −
2025-05	xboing-python	301	42,817	17,129
2025-06	xboing-python	1	7	651

Month	Repo	Commits	Lines +	Lines −
2025-11	xboing-python	5	17,421	2,024
2025-11	xboing-c	2	39,541	39
2026-02	xboing-c	162	64,778	18,366
2026-03	xboing-c	52	8,855	773
2026-04	xboing-c	8	5,229	889
2026-05	xboing-c	40	24,458	2,866
2026-06	xboing-python	3	4,323	1,262

The two projects never had concurrent active development. `xboing-python`'s burst ended in May 2025; `xboing-c`'s began in November 2025. The February 2026 spike in `xboing-c` (162 commits, +64K lines) represents the SDL2 rewrite of the core game systems.

4 AI-Assisted Development

Both projects are entirely AI-written. `xboing-c` was written with Claude Code (Opus 4/4.5/4.6). `xboing-python` was written with Cursor. In May 2025, Cursor's available models were Claude 3.5 Sonnet v2 (the default coding model) and GPT-4o. The difference between the two projects is in model generation, tooling maturity, and workflow conventions.

4.1 Co-Authorship Tags

Commits containing a `Co-Authored-By: Claude` tag or `Generated with Claude Code` marker:

Repo	Total	Tagged	Tag Rate
xboing-python	310	5	1.6%
xboing-c	264	202	76.5%

Interpretation. The low tag rate on `xboing-python` reflects the use of Cursor, which does not add co-authorship tags. The 5 tagged commits (November 2025 and June 2026) correspond to the few occasions where Claude Code was used on the Python repo. `xboing-c` used Claude Code exclusively, where tagging is standard.

4.2 Tagged Commits by Month

Month	xboing-c (tagged)	xboing-python (tagged)
2025-05	—	0 of 301
2025-11	1	2
2026-02	82	—
2026-03	44	—
2026-04	33	—
2026-05	42	—
2026-06	—	3

5 Claude Model Availability Windows

The following table maps Claude model releases against the development timelines of both projects. Release dates are approximate based on public announcements.

Date	Model	Project Activity
2024-06	**Sonnet 3.5**	Neither project started.
2024-10	**Sonnet 3.5 v2**, **Haiku 3.5**	Neither project started.

Date	Model	Project Activity
2025-02	**Sonnet 3.5 v2** (Claude Code early access)	Neither project started. Claude Code in limited preview.
2025-04	**Sonnet 3.5 v2** (Claude Code GA)	`xboing-python` starts (May 3) using Cursor. 301 commits in May. Cursor's models: Claude 3.5 Sonnet v2 (default) and GPT-4o.
2025-06	**Sonnet 4 (preview)**	`xboing-python` goes dormant (1 commit in June).
2025-10	**Sonnet 3.6**	Both projects dormant.
2025-11	**Sonnet 3.6**	`xboing-c` baseline commit (Nov 26). `xboing-python` brief activity (5 commits).
2025-12	**Opus 4**	Both projects dormant. Opus 4 is the first model with strong agentic coding capability.
2026-01	**Sonnet 4.5**, **Haiku 4.5**	`xboing-c` dormant between baseline and main development phase.
2026-02	**Opus 4** (dominant coding model)	`xboing-c` primary development burst begins. 162 commits, 82 Claude co-authored. The SDL2 rewrite of core game systems happens under Opus 4.
2026-03	**Opus 4.5**	`xboing-c` continues. 52 commits, 44 Claude co-authored.
2026-04	**Opus 4.5**	`xboing-c` lower intensity. 8 commits, 33 Claude co-authored (includes branches).
2026-05	**Opus 4.6**, **Sonnet 4.6**	`xboing-c` second surge. 40 commits, 42 Claude co-authored. Savegame system, dialogue, visual polish.
2026-06	**Opus 4.6**	`xboing-python` brief activity (3 commits, dependency upgrades + PR cleanup).

5.1 Key Timing Observations

1. **Both projects are entirely AI-written.** The difference is not whether AI was used, but which model generation and tooling were available.

2. `xboing-python` **was built with Cursor and Sonnet 3.5 v2 / GPT-4o.** The May 2025 burst used Cursor with Claude 3.5 Sonnet v2 as the primary coding model. Cursor in this period offered inline completion, chat, and composer — but not the agentic autonomy of later Claude Code (no subagents, no PR automation, no hook-based workflows).

3. `xboing-c` **was built with Opus 4/4.5/4.6 and mature Claude Code.** The entire SDL2 rewrite (Feb–May 2026) coincides with the most capable Claude models to date and a substantially more mature tooling ecosystem.

4. **Model capability inflection.** The jump from Sonnet 3.5 v2 (May 2025) to Opus 4+ (Feb–May 2026) represents a step change in coding capability: longer context, better tool use, stronger multi-file reasoning, and more reliable code generation. This capability difference directly impacts output volume and quality per session.

5. **Tooling paradigm shift.** Beyond model capability, the tooling paradigm changed between the two projects. Cursor (May 2025) is an IDE with AI-assisted editing — the developer drives, the AI assists. Claude Code (Feb–May 2026) is an agentic CLI where the AI drives autonomously — creating branches, running tests, reviewing PRs, resolving comments, and merging. Features

like automated PR review loops, merge automation, subagents, and multi-agent orchestration were available only for `xboing-c`.

6 Investment Comparison

6.1 Effort Density

Metric	xboing-python	xboing-c
Active development days	24	30
Commits per active day	12.9	8.8
Lines added per active day	2,690	4,762
PRs per active day	0.75	4.6
Test functions per active day	8.4	37.2

Interpretation. `xboing-c` produces 1.8× more lines per active day and 4.4× more test functions per active day. Both projects are entirely AI-written, so the productivity difference reflects the model and tooling generations: Opus 4+ with agentic Claude Code (subagents, PR automation, hooks) vs Sonnet 3.5 v2 / GPT-4o with Cursor (assisted editing).

6.2 Quality Investment

Metric	xboing-python	xboing-c
Test:source line ratio	0.46:1	1.09:1
Test functions	202	1,115
Fuzz targets	0	4
Golden screenshot tests	No	Yes
Integration test suites	1	6
Merged PRs (review gates)	18	137

`xboing-c` has more than double the test-to-source ratio and 5.5× more test functions. It includes fuzz testing for all I/O parsers and golden screenshot tests for visual fidelity. Every change goes through a PR with automated CI and reviewer sign-off (137 merged PRs vs 18).

6.3 Feature Output

Metric	xboing-python	xboing-c
Game modes implemented	4 of 16	16 of 16
Block types with behavior	∼12 of 30	30 of 30
Visual effects	0 of 5	5 of 5
Persistence systems	0 of 3	3 of 3
Specials with behavior	2 of 8	8 of 8

7 Synthesis

1. **Both projects are entirely AI-written; the model and tooling generation is the variable.** The output difference — 2.75× more code, 5.5× more tests, 4× more game modes — is not explained by one project using AI and the other not. Both are AI-written. The difference is Cursor + Sonnet 3.5 v2 (May 2025) vs Claude Code + Opus 4/4.5/4.6 (Feb–May 2026).

2. **Active development days are comparable.** `xboing-python`: 24 active days. `xboing-c`: 30 active days. The 25% difference in active days does not explain the 2–5× difference in output. The multiplier is model capability and tooling maturity.

3. **Agentic vs assisted is the tooling divide.** Cursor (2025) assists a developer who drives the workflow. Claude Code (2026) drives autonomously — creating branches, writing code, running quality gates, opening PRs, addressing review comments, and merging. The 137 merged PRs on

`xboing-c` vs 18 on `xboing-python` reflect this paradigm difference: automated PR creation, review loops, and merge automation were not part of the Cursor workflow.

4. **The PR workflow gap is a consequence of tooling, not discipline.** `xboing-python` pushed directly to master (301 commits in May 2025) because Cursor does not automate branch/PR/merge workflows. `xboing-c` used PRs for every change because Claude Code automates the entire cycle. The later project's higher test coverage and review discipline follow from the tooling.

5. **The Python port's foundations are sound.** Despite the completeness gap, `xboing-python` has a clean MVC architecture, strict type checking (mypy + pyright in strict mode), protocol-based design, and dependency injection. Bringing the Python port to parity would require implementing the missing game systems, not rearchitecting what exists.